

1 Introducción.....	4
Contexto del proyecto.....	4
Objetivo general.....	4
2 Descripción general del sistema.....	4
Funcionalidad de la API.....	4
Arquitectura general.....	5
Funcionamiento del flujo de datos.....	5
Organización modular del backend.....	6
Tecnologías utilizadas.....	6
Python.....	7
Django.....	7
Django REST Framework.....	7
SQLite.....	7
Stripe.....	7
Git/GitHub.....	8
Postman.....	8
3 Metodología de desarrollo.....	8
Organización por hitos.....	8
Planificación semanal.....	9
Uso de Trello para la gestión del proyecto.....	9
Gestión del repositorio GitHub.....	11
4 Análisis de requisitos.....	11
Requisitos funcionales.....	11
Gestión de proveedores.....	11
Registro de transacciones.....	12
Gestión de incidencias.....	12
Requisitos técnicos.....	12
Arquitectura REST.....	12
Seguridad.....	13
Validaciones.....	13
Estándares PEP8.....	13
Modularidad.....	14
5 Diseño de la base de datos.....	14
Modelo entidad-relación (ERD).....	14
Explicación de entidades.....	15
Provider.....	15
Campos principales.....	15
Función dentro del sistema.....	16
Transaction.....	16
Campos principales.....	16
Estados posibles.....	17
Función dentro del sistema.....	17
Incidence.....	17

Campos principales.....	17
Tipos de incidencias.....	17
Función dentro del sistema.....	18
6 Arquitectura del proyecto.....	18
Estructura de carpetas.....	18
Apps creadas.....	19
Configuración principal.....	20
Variables de entorno.....	20
Flujo de peticiones.....	20
1. Cliente (Postman).....	20
2. Sistema de rutas.....	20
3. Views.....	20
4. Serializers.....	21
5. Modelos y ORM.....	21
6. Integración con Stripe.....	21
7 Implementación del backend.....	22
Modelos.....	22
Provider.....	22
Campos principales.....	22
Validaciones implementadas.....	22
Relación con otros modelos.....	22
Transaction.....	23
Campos principales.....	23
Estados de transacción.....	23
Validaciones implementadas.....	23
Relación con otros modelos.....	24
Incidence.....	24
Campos principales.....	24
Tipos de incidencias.....	24
Validaciones implementadas.....	24
Serializers.....	25
ProviderSerializer.....	25
Campos serializados.....	25
Validaciones implementadas.....	25
Validación del nombre.....	25
Validación del entorno.....	25
TransactionSerializer.....	26
Campos serializados.....	26
Validaciones implementadas.....	26
Validación del importe.....	26
Validación de la moneda.....	26
Validación del estado del pago.....	26
Validación general.....	27
IncidenceSerializer.....	27

Campos serializados.....	27
Validaciones implementadas.....	27
Validación de la descripción.....	27
Validación del tipo de incidencia.....	27
Validación general.....	28
Views.....	28
Gestión de proveedores.....	28
Funcionalidades implementadas.....	28
Gestión de transacciones.....	29
TransactionModelViewSet.....	29
Creación de pagos con Stripe.....	29
Flujo de creación del pago.....	29
Datos almacenados.....	29
Manejo de errores.....	30
Actualización automática mediante Webhooks.....	30
Funcionamiento del webhook.....	30
Flujo del webhook.....	30
Gestión de devoluciones.....	30
Proceso de devolución.....	31
Registro de incidencias.....	31
Gestión de incidencias.....	31
Integración con Stripe.....	31
Rutas y endpoints.....	32
8 Pruebas realizadas.....	33
Pruebas con Postman.....	33
Validación de errores.....	34
Errores controlados.....	34
9 Problemas encontrados y soluciones.....	34
Problemas relacionados con Stripe.....	34
Solución aplicada.....	35
Problemas con los webhooks.....	35
Solución aplicada.....	35
Problemas con validaciones de datos.....	35
Solución aplicada.....	35
Problemas con las devoluciones.....	36
Solución aplicada.....	36
Aprendizajes obtenidos.....	36
10 Conclusiones.....	36
Objetivos alcanzados.....	37
Aprendizajes obtenidos.....	37
Posibles mejoras futuras.....	38
11 Bibliografía.....	38

1 Introducción

Contexto del proyecto

El crecimiento del comercio electrónico ha provocado que las plataformas digitales necesitan sistemas de pago cada vez más seguros, rápidos y eficientes. Dentro de este entorno, las pasarelas de pago desempeñan un papel fundamental, ya que permiten procesar transacciones electrónicas entre clientes y empresas de forma automatizada.

Aunque existen múltiples proveedores de pago en el mercado, durante el desarrollo de este proyecto se ha trabajado exclusivamente con Stripe, una de las plataformas más utilizadas actualmente debido a su facilidad de integración, seguridad y amplia documentación para desarrolladores.

El proyecto consiste en el desarrollo de una API REST utilizando Django y Django REST Framework (DRF), orientada a la gestión de transacciones de pago mediante Stripe. La aplicación permite registrar proveedores, crear y administrar transacciones, gestionar incidencias relacionadas con los pagos y proteger el acceso a los endpoints mediante autenticación.

Además, el sistema ha sido diseñado siguiendo una arquitectura modular y escalable, aplicando buenas prácticas de programación backend, control de versiones con GitHub y estándares de calidad como PEP8.

Objetivo general

El objetivo principal del proyecto es desarrollar un backend seguro y parametrizable para la gestión de pasarelas de pago mediante una API RESTful.

La aplicación debe permitir:

- Registrar y administrar diferentes proveedores de pago.
- Gestionar transacciones de pago y su estado.
- Registrar incidencias relacionadas con errores o devoluciones.
- Consultar historiales de transacciones aplicando filtros específicos.
- Además, el proyecto busca aplicar buenas prácticas de desarrollo backend utilizando Django REST Framework, siguiendo estándares de calidad como PEP8, el uso de GitHub para el control de versiones y una estructura modular del proyecto.

2 Descripción general del sistema

Funcionalidad de la API

Se ha desarrollado una API que permite hacer llamadas al back con la finalidad de procesar pagos mediante distintas pasarelas.

Durante este desarrollo se ha centrado exclusivamente en el uso de la pasarela de Stripe, no obstante se ha tenido en cuenta que más adelante se pueden añadir nuevas pasarelas al proyecto.

El desarrollo de la API permite los siguientes puntos:

- Gestión de proveedores
- Registro de pagos
- Actualización de los estados de los pagos
- Creación de incidencias

Arquitectura general

La arquitectura del sistema ha sido diseñada siguiendo el modelo de una API REST desarrollada con Django REST Framework (DRF), permitiendo separar de forma clara la lógica de negocio, la gestión de datos y la comunicación con servicios externos.

El backend actúa como intermediario entre el cliente y la plataforma de pagos Stripe, encargándose de validar las solicitudes, procesar las operaciones y almacenar la información relacionada con las transacciones realizadas.

Durante el desarrollo del proyecto, las pruebas y peticiones a la API se han realizado mediante Postman, herramienta utilizada para enviar solicitudes HTTP a los diferentes endpoints implementados. Una vez recibida una petición, el backend procesa la información y se comunica con la API de Stripe para gestionar el pago correspondiente. Finalmente, el sistema registra en la base de datos toda la información relacionada con la transacción y su estado.

El flujo principal de funcionamiento del sistema es el siguiente:

Postman → API REST (Django REST Framework) → Stripe API → Base de datos SQLite

Funcionamiento del flujo de datos

- Postman realiza una petición HTTP
El usuario envía una solicitud a la API utilizando métodos como GET, POST, PUT o DELETE para gestionar proveedores, transacciones o incidencias.
- La API REST procesa la petición
Django REST Framework recibe la solicitud y ejecuta la lógica correspondiente mediante las vistas y serializadores definidos en el proyecto. Durante este proceso se validan los datos recibidos y se comprueba la autenticación del usuario.
- Comunicación con Stripe
Cuando se requiere procesar un pago, el backend se conecta con la API de Stripe utilizando claves privadas configuradas en el sistema. Stripe se encarga de simular y gestionar el procesamiento seguro de la transacción.
- Registro en la base de datos

Tras recibir la respuesta de Stripe, el sistema almacena la información de la operación en SQLite, incluyendo datos como:

- importe
- moneda
- identificador de la transacción
- estado del pago
- fecha de creación

Organización modular del backend

El proyecto sigue una estructura modular propia de Django, separando las distintas responsabilidades en componentes independientes. Esta organización facilita:

- Mantenimiento del código
- Escalabilidad del sistema
- Reutilización de componentes
- Claridad del desarrollo

Los principales componentes del backend son:

- Models: representan las entidades de la base de datos.
- Serializers: convierten los datos entre objetos Python y formato JSON.
- Views: contienen la lógica de negocio y gestionan las peticiones HTTP.
- URLs: definen las rutas de acceso a los endpoints de la API.
- Settings: almacenan la configuración general del proyecto y carga las credenciales de Stripe almacenadas en un archivo .env

Además, la arquitectura REST utilizada permite que el backend pueda ser consumido por diferentes tipos de clientes, como aplicaciones web, aplicaciones móviles o servicios externos.

Tecnologías utilizadas

En el desarrollo del proyecto se han utilizado diferentes tecnologías y herramientas orientadas al desarrollo backend, la gestión de bases de datos, el control de versiones y las pruebas de la API. Cada una de ellas cumple una función específica dentro de la arquitectura del sistema.

Tecnología	Versión
Python	3.14.0
Django	6.0.5
Django REST Framework	3.17.1
Stripe	15.1.0

Python

Python ha sido el lenguaje de programación principal utilizado para el desarrollo del backend. Se trata de un lenguaje interpretado y multiparadigma ampliamente utilizado en el desarrollo web debido a su sintaxis sencilla, legibilidad y gran ecosistema de librerías.

Django

Django es el framework principal utilizado para desarrollar la aplicación backend. Este framework permite acelerar el desarrollo gracias a su estructura modular y a las funcionalidades integradas que proporciona, como:

- Sistema de rutas
- ORM para bases de datos
- Gestión de usuarios
- Administración automática
- Medidas de seguridad integradas

Además, Django facilita la organización del proyecto siguiendo buenas prácticas de desarrollo.

Django REST Framework

Django REST Framework se ha utilizado para la construcción de la API REST del proyecto. Esta extensión de Django permite desarrollar endpoints de forma eficiente mediante herramientas como:

- Serializers
- Views
- Autenticación
- Permisos
- Validaciones automáticas

Gracias a DRF, la API puede intercambiar información en formato JSON de manera estructurada y segura.

SQLite

SQLite ha sido la base de datos utilizada durante el desarrollo del proyecto. Se trata de un sistema de gestión de bases de datos ligero y fácil de configurar, integrado por defecto en Django.

Stripe

Stripe ha sido la plataforma de pagos utilizada en el proyecto para simular y gestionar transacciones electrónicas.

La integración con Stripe permite:

- Crear pagos

- Validar operaciones
- Gestionar respuestas del proveedor
- Almacenar el estado de las transacciones en la base de datos

Para garantizar la seguridad, las claves privadas y credenciales sensibles se almacenan mediante variables de entorno utilizando un archivo .env

Git/GitHub

Git se ha utilizado como sistema de control de versiones para gestionar el desarrollo del proyecto y registrar los cambios realizados durante cada fase de implementación.

GitHub ha servido como plataforma de alojamiento del repositorio remoto, permitiendo mantener un historial de commits y una correcta organización del código fuente.

Postman

Postman se ha utilizado como herramienta principal para realizar pruebas sobre los endpoints de la API.

Gracias a esta herramienta ha sido posible:

- Enviar peticiones HTTP
- Validar respuestas
- Comprobar códigos de estado
- Verificar el correcto funcionamiento de las operaciones CRUD implementadas en el sistema.

3 Metodología de desarrollo

Para el desarrollo del proyecto se ha seguido una metodología de trabajo organizada por fases, permitiendo estructurar correctamente las tareas y controlar el progreso de implementación del backend.

La planificación del proyecto se ha basado en una metodología inspirada en SCRUM, dividiendo el desarrollo en diferentes hitos y organizando las tareas según su estado de realización.

Con el objetivo de mejorar la gestión del trabajo, se ha utilizado la herramienta Trello como sistema de seguimiento de tareas y planificación del proyecto.

Organización por hitos

El desarrollo del proyecto se dividió en varias fases principales:

- Investigación y documentación inicial
- Diseño de la base de datos

- Implementación de la arquitectura backend
- Integración con Stripe
- Desarrollo de endpoints
- Validaciones
- Pruebas
- Documentación final

Cada fase permitió avanzar progresivamente en la construcción del sistema, manteniendo una organización clara de los objetivos del proyecto.

Planificación semanal

La planificación se realizó distribuyendo las tareas según el nivel de prioridad y dependencia entre componentes.

Durante las primeras etapas del proyecto se trabajó principalmente en:

- estudio de Django REST Framework
- documentación oficial de Stripe
- y diseño de la arquitectura de la base de datos

Posteriormente se desarrollaron:

- Modelos
- Serializers
- Vistas
- Integración con Stripe Checkout
- Sistema de webhooks
- Gestión de devoluciones

Finalmente se realizaron:

- Pruebas funcionales mediante Postman
- Validaciones
- Corrección de errores
- Elaboración de la memoria técnica

Uso de Trello para la gestión del proyecto

Para organizar el flujo de trabajo se utilizó Trello como herramienta de gestión visual de tareas.

El tablero se estructuró utilizando diferentes columnas que representaban el estado de cada tarea:

- Lista de tareas

- En proceso
- Testing
- Hecho

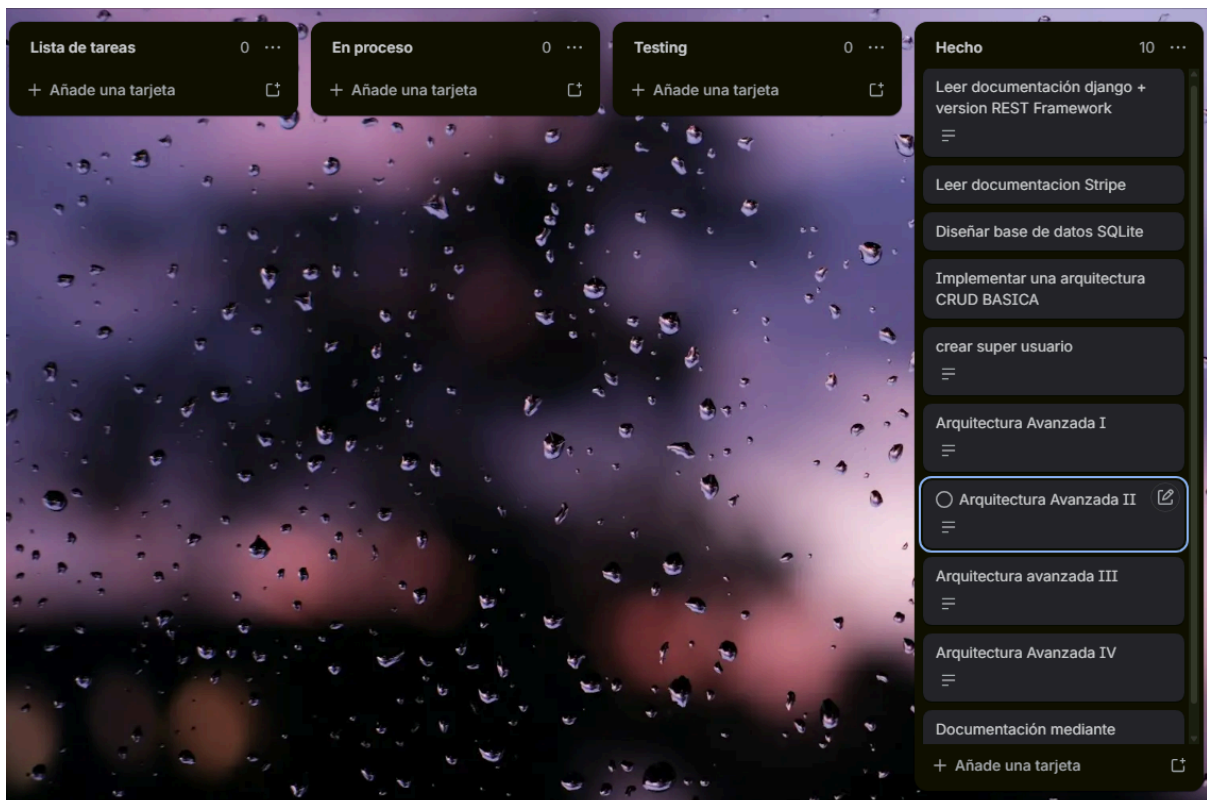
Cada tarjeta incluía funcionalidades o tareas específicas relacionadas con el desarrollo del backend, como:

- Lectura de documentación de Django REST Framework
- Integración de Stripe
- Diseño de la base de datos
- Implementación de arquitectura CRUD
- Desarrollo de funcionalidades avanzadas

Este sistema permitió:

- controlar el progreso del proyecto,
- organizar prioridades,
- visualizar tareas pendientes,
- y mantener una planificación más clara durante todo el desarrollo.

Además, Trello facilitó una organización más cercana a entornos reales de trabajo colaborativo y metodologías ágiles.



Gestión del repositorio GitHub

El control de versiones del proyecto se realizó utilizando Git y GitHub.

Durante el desarrollo se efectuaron commits regulares para registrar la evolución del sistema y mantener un historial organizado de cambios. Esto permitió:

- Controlar versiones del código,
- Recuperar cambios anteriores,
- Mantener una trazabilidad completa del desarrollo.

El repositorio también se utilizó para almacenar:

- Código fuente,
- Documentación,
- Archivo README.md,
- Configuración general del proyecto.

4 Análisis de requisitos

Durante el desarrollo del proyecto se realizó un análisis previo de los requisitos funcionales y técnicos necesarios para garantizar el correcto funcionamiento de la API REST.

El sistema debía ser capaz de gestionar pagos mediante Stripe, almacenar información relacionada con las transacciones y ofrecer mecanismos de control y seguridad adecuados para una aplicación backend.

Requisitos funcionales

Los requisitos funcionales definen las operaciones y funcionalidades que el sistema debe ofrecer al usuario

Gestión de proveedores

La API debía permitir registrar y gestionar proveedores de pago dentro del sistema.

Aunque la arquitectura fue diseñada para soportar múltiples proveedores, durante el desarrollo únicamente se implementó Stripe como plataforma principal de procesamiento de pagos.

El sistema permite:

- Crear proveedores
- Consultar información de proveedores
- Modificar configuraciones
- Eliminar registros existentes.

Registro de transacciones

El sistema debía permitir registrar todas las transacciones realizadas mediante Stripe.

Cada transacción almacena información relacionada con:

- importe
- moneda
- proveedor asociado
- estado del pago
- identificadores generados por Stripe
- fechas de creación

El objetivo principal es mantener trazabilidad completa sobre cada operación realizada en la plataforma.

Gestión de incidencias

La API debía incorporar un sistema para registrar incidencias relacionadas con las transacciones.

Estas incidencias permiten documentar:

- devoluciones
- errores de conexión
- problemas de autenticación
- pagos fallidos

Gracias a ello, el sistema puede realizar un seguimiento detallado de problemas ocurridos durante el procesamiento de pagos.

Requisitos técnicos

Los requisitos técnicos definen las tecnologías, estándares y criterios de calidad utilizados durante el desarrollo del proyecto.

Arquitectura REST

La aplicación debía seguir una arquitectura RESTful para facilitar la comunicación entre cliente y servidor mediante peticiones HTTP.

Para ello se utilizaron:

- Endpoints organizados
- Métodos HTTP (GET, POST, PUT, DELETE)

- Respuestas en formato JSON
- Códigos de estado HTTP adecuados

Esto permite que la API pueda integrarse fácilmente con aplicaciones externas o futuros frontends.

Seguridad

El sistema debía implementar medidas básicas de seguridad para proteger la información y las operaciones realizadas.

Entre las medidas implementadas destacan:

- Uso de variables de entorno mediante .env
- Protección de claves privadas de Stripe
- Validación de webhooks
- Validación de datos recibidos
- Control de acceso a endpoints

Estas medidas ayudan a prevenir accesos indebidos y manipulaciones externas.

Validaciones

El proyecto debía garantizar la integridad de los datos almacenados mediante validaciones automáticas y personalizadas.

Las validaciones se implementaron tanto en:

- Modelos (clean()),
- Como en serializers de Django REST Framework.

Estas comprobaciones permiten:

- Evitar datos inválidos,
- Controlar tamaños máximos,
- Validar estados permitidos,
- Verificar relaciones entre entidades

Estándares PEP8

El desarrollo del proyecto se realizó siguiendo las recomendaciones de estilo definidas por PEP8 para Python.

Esto permitió mantener:

- Código más limpio
- Legible
- Mantenible
- Organizado

Además, se utilizaron nombres descriptivos para clases, funciones y variables con el objetivo de mejorar la comprensión del código

Modularidad

El backend fue desarrollado siguiendo una estructura modular basada en aplicaciones y componentes independientes de Django.

La lógica del sistema se separó en:

- Modelos
- Serializers
- Vistas
- Rutas
- Configuración general

Esta organización facilita:

- Mantenimiento
- Escalabilidad
- Reutilización de código
- Futuras ampliaciones del proyecto

5 Diseño de la base de datos

La base de datos del proyecto ha sido diseñada utilizando SQLite como sistema de almacenamiento relacional.

El objetivo principal del diseño ha sido estructurar correctamente la información relacionada con:

- Proveedores de pago
- Transacciones
- Incidencias

Para ello, se han definido diferentes modelos relacionales mediante Django ORM, permitiendo mantener integridad entre tablas y facilitar la gestión de datos dentro del sistema.

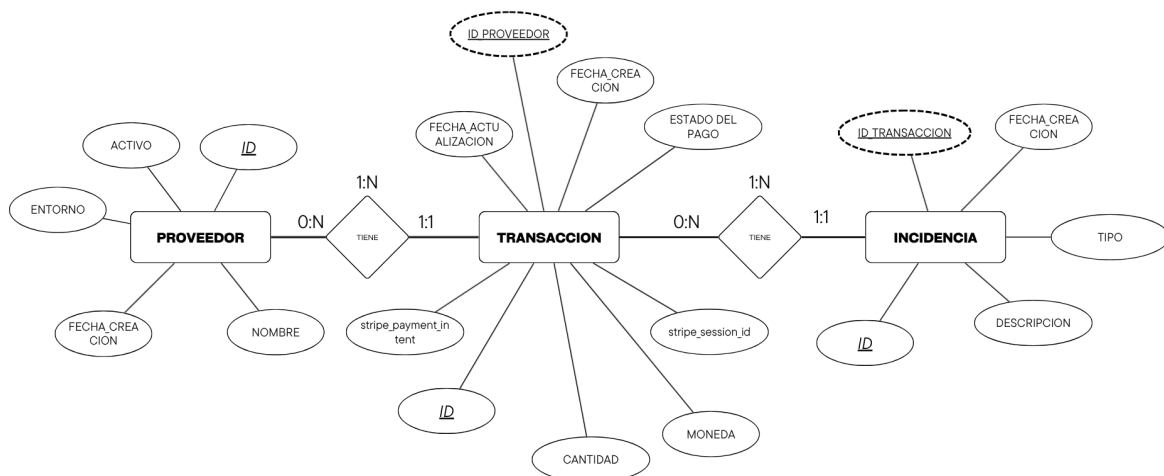
Modelo entidad-relación (ERD)

La estructura de la base de datos se basa en tres entidades principales:

- Provider
- Transaction
- Incidence

Estas entidades mantienen relaciones jerárquicas que permiten representar el flujo completo de una operación de pago.

El modelo relacional del sistema es el siguiente:



- Un proveedor puede tener múltiples transacciones.
- Una transacción pertenece únicamente a un proveedor.
- Una transacción puede registrar múltiples incidencias.
- Una incidencia pertenece únicamente a una transacción.

Explicación de entidades

Provider

La entidad Provider almacena la información relacionada con los proveedores de pago disponibles en el sistema.

Aunque la arquitectura está preparada para soportar múltiples proveedores, durante el desarrollo únicamente se ha trabajado con Stripe.

Campos principales

Campo	Tipo	Descripción
id	INTEGER	Identificador único
name	VARCHAR(100)	Nombre del proveedor
environment	VARCHAR	Entorno (test o prod)
active	BOOLEAN	Estado del proveedor
creation_date	DATETIME	Fecha de creación

Función dentro del sistema

Esta entidad permite:

- Identificar proveedores
- Controlar entornos de trabajo
- Habilitar o deshabilitar proveedores de pago

Transaction

La entidad Transaction representa las operaciones de pago realizadas mediante Stripe.

Cada transacción almacena información relacionada con el importe, estado del pago y datos devueltos por Stripe.

Campos principales

Campo	Tipo	Descripción
id	INTEGER	Identificador único
amount	DECIMAL	Importe del pago
currency	VARCHAR(10)	Moneda utilizada
payment_state	VARCHAR	Estado de la transacción
creation_date	DATETIME	Fecha de creación
update_date	DATETIME	Fecha de actualización
stripe_session_id	VARCHAR(255)	ID de sesión Stripe
stripe_payment_intent	VARCHAR(255)	Payment Intent de Stripe
id_proveedor_id	BIGINT	Relación con Provider

Estados posibles

El sistema permite gestionar varios estados:

- pending
- completed
- failed
- refunded

Función dentro del sistema

La entidad permite:

- Registrar pagos
- Controlar estados
- Almacenar datos de Stripe
- Mantener historial de operaciones

Incidence

La entidad Incidence almacena incidencias relacionadas con las transacciones registradas en el sistema.

Su función principal es mantener un registro de errores o eventos ocurridos durante el procesamiento de pagos.

Campos principales

Campo	Tipo	Descripción
id	INTEGER	Identificador único
description	VARCHAR(500)	Descripción de la incidencia
type	VARCHAR	Tipo de incidencia
creation_date	DATETIME	Fecha de creación
id_transaction_id	BIGINT	Relación con Transaction

Tipos de incidencias

El sistema contempla:

- nonpayment
- connection_error
- devolution
- timeout

- authentication_error

Función dentro del sistema

Esta entidad permite:

- Registrar errores
- Almacenar devoluciones
- Controlar incidencias técnicas
- Mejorar la trazabilidad del sistema

6 Arquitectura del proyecto

La arquitectura del proyecto ha sido desarrollada utilizando Django y Django REST Framework siguiendo una estructura modular y organizada.

El objetivo principal de esta arquitectura es separar responsabilidades dentro del sistema, facilitando:

- Mantenimiento del código
- Escalabilidad
- Reutilización de componentes
- Claridad en el desarrollo backend

El proyecto se organiza mediante diferentes componentes que trabajan conjuntamente para gestionar la comunicación entre el cliente, Stripe y la base de datos.

Estructura de carpetas

El proyecto sigue la estructura estándar de Django, separando la configuración general del sistema de la lógica específica de la aplicación

Esta organización permite mantener separadas:

- Configuraciones globales
- Lógica de negocio
- Rutas de la API

Validaciones de datos

```
> docs
▼ Pasarelas
  > .pytest_cache
  ▼ pasarela
    > __pycache__
    ▼ api
      > __pycache__
      • __init__.py
      • router.py
      • serializers.py
      • views.py
    > migrations
    > tests
    • __init__.py
    • admin.py
    • apps.py
    • models.py
    • serializers.py
    • views.py
  ▼ Pasarelas
    > __pycache__
    • __init__.py
    • asgi.py
    • settings.py
    • urls.py
    • wsgi.py
  > static
  • .env
  • .gitignore
  • db.sqlite3
  • manage.py
  • pytest.ini
```

Apps creadas

El sistema se ha desarrollado principalmente mediante la aplicación pasarela, encargada de contener toda la lógica relacionada con la gestión de pagos y transacciones.

Dentro de esta aplicación se organizan:

- Modelos de base de datos
- Serializers

- Vistas
- Endpoints
- Lógica de integración con Stripe

La carpeta `api/` se utiliza para separar específicamente los componentes relacionados con Django REST Framework.

Configuración principal

La configuración global del proyecto se gestiona desde el archivo `settings.py`.

En este archivo se configuran aspectos fundamentales como:

- Aplicaciones instaladas
- Base de datos SQLite
- Middleware
- Configuración REST Framework
- Variables relacionadas con Stripe

Variables de entorno

Para mejorar la seguridad del sistema, las credenciales sensibles no se almacenan directamente en el código fuente.

Las claves privadas de Stripe se guardan en un archivo `.env` y posteriormente se cargan desde `settings.py`.

Flujo de peticiones

El sistema sigue una arquitectura cliente-servidor basada en peticiones HTTP.

1. Cliente (Postman)

Las peticiones se realizan mediante Postman utilizando métodos HTTP:

- GET
- POST
- PUT
- PATCH
- DELETE

2. Sistema de rutas

Las URLs definidas en Django reciben las peticiones y las redirigen hacia la vista correspondiente.

Cada endpoint se encuentra asociado a una funcionalidad específica de la API.

3. Views

Las vistas contienen la lógica principal del backend.

Se encargan de:

- Procesar peticiones
- Validar operaciones
- Interactuar con Stripe
- Devolver respuestas HTTP

El proyecto utiliza:

- ModelViewSet
- Vistas personalizadas
- Funciones decoradas con `@api_view`

4. Serializers

Los serializers convierten los datos entre:

- Objetos Python
- Modelos Django
- Formato JSON

Además, realizan validaciones para garantizar la integridad de los datos recibidos.

5. Modelos y ORM

Los modelos representan las tablas de la base de datos utilizando Django ORM.

Gracias al ORM es posible:

- Realizar consultas sin escribir SQL directamente
- Gestionar relaciones
- Simplificar operaciones CRUD

6. Integración con Stripe

Cuando una operación requiere procesar pagos:

- El backend se comunica con Stripe mediante su API oficial
- Crea sesiones de pago
- Valida webhooks
- Gestiona devoluciones

Stripe devuelve información que posteriormente se almacena en la base de datos.

7 Implementación del backend

Modelos

Para la implementación del backend se han utilizado modelos de Django ORM con el objetivo de representar las diferentes entidades necesarias para el funcionamiento de la API.

Los modelos permiten gestionar la persistencia de los datos en la base de datos SQLite y establecer relaciones entre proveedores, transacciones e incidencias. Además, se han implementado validaciones personalizadas mediante el método `clean()` para garantizar la integridad de los datos y evitar posibles manipulaciones o inconsistencias.

El sistema está compuesto por tres modelos principales

Provider

El modelo Provider representa los proveedores de pago disponibles en el sistema. Aunque la arquitectura está preparada para soportar múltiples plataformas, durante el desarrollo del proyecto únicamente se ha utilizado Stripe como proveedor principal.

Campos principales

Campo	Tipo	Descripción
name	CharField	Nombre del proveedor
environment	CharField	Entorno de trabajo (test o prod)
active	BooleanField	Estado de activación del proveedor
creation_date	DateTimeField	Fecha de creación automática

Validaciones implementadas

El modelo incluye varias validaciones personalizadas:

- Comprobación de nombre vacío
- Límite máximo de caracteres
- Verificación de nombres duplicados
- Validación de los entornos permitidos

Estas validaciones ayudan a proteger el sistema frente a datos manipulados o configuraciones incorrectas.

Relación con otros modelos

El modelo Provider mantiene una relación uno-a-muchos con el modelo Transaction, ya que un proveedor puede gestionar múltiples transacciones.

Transaction

El modelo Transaction almacena toda la información relacionada con los pagos procesados mediante Stripe.

Cada transacción queda asociada a un proveedor y contiene información sobre el importe, estado del pago y datos devueltos por Stripe durante el procesamiento de la operación.

Campos principales

Campo	Tipo	Descripción
id_proveedor	ForeignKey	Relación con el proveedor
stripe_session_id	CharField	Identificador de sesión de Stripe
amount	DecimalField	Importe de la transacción
currency	CharField	Tipo de moneda utilizada
payment_state	CharField	Estado del pago
stripe_payment_intent	CharField	Identificador del pago en Stripe
creation_date	DateTimeField	Fecha de creación
update_date	DateTimeField	Fecha de actualización

Estados de transacción

El sistema permite gestionar diferentes estados para cada pago:

- pending
- completed
- failed
- refunded

Esto facilita el seguimiento completo del ciclo de vida de cada transacción.

Validaciones implementadas

Las validaciones incluidas en el modelo permiten:

- evitar importes negativos,
- comprobar la existencia del proveedor asociado,
- validar el formato de la moneda,

- restringir estados de pago inválidos.

Además, se verifican los datos recibidos para impedir posibles manipulaciones externas.

Relación con otros modelos

El modelo Transaction mantiene una relación uno-a-muchos con Incidence, ya que una transacción puede registrar múltiples incidencias a lo largo de su procesamiento.

Incidence

El modelo Incidence permite registrar errores o incidencias relacionadas con las transacciones realizadas en el sistema.

Su objetivo es proporcionar trazabilidad sobre problemas ocurridos durante el procesamiento de pagos, facilitando la monitorización y el control de errores.

Campos principales

Campo	Tipo	Descripción
id_transaction	ForeignKey	Relación con la transacción
description	CharField	Descripción de la incidencia
type	CharField	Tipo de incidencia
creation_date	DateTimeField	Fecha de registro

Tipos de incidencias

El sistema contempla diferentes tipos de incidencias:

- nonpayment
- connection_error
- devolution
- timeout
- authentication_error

Validaciones implementadas

El modelo incluye validaciones para:

- Verificar la existencia de la transacción asociada
- Controlar la longitud máxima de la descripción
- Validar los tipos de incidencia permitidos

Estas comprobaciones ayudan a mantener la consistencia de la información almacenada en la base de datos.

Serializers

Para la comunicación entre la API y los clientes externos se han utilizado serializers de Django REST Framework.

Los serializers permiten convertir los modelos de Django en formato JSON y viceversa, facilitando el intercambio de información entre el backend y aplicaciones externas como Postman. Además, se han implementado validaciones personalizadas para controlar la integridad de los datos recibidos antes de almacenarlos en la base de datos.

ProviderSerializer

El serializer ProviderSerializer se encarga de gestionar la serialización y validación de los datos relacionados con los proveedores de pago.

Campos serializados

Campo

id

name

environment

active

creation_date

Validaciones implementadas

Se han desarrollado validaciones específicas para garantizar la consistencia de los datos:

Validación del nombre

Se comprueba que:

- el nombre no esté vacío,
- no contenga únicamente espacios,
- y no supere el límite máximo de caracteres permitido.

Validación del entorno

El sistema restringe los valores permitidos para el entorno del proveedor:

- dev

- test
- prod

TransactionSerializer

El serializer TransactionSerializer permite gestionar las transacciones de pago procesadas mediante Stripe.

Su función principal es validar la información recibida antes de crear o actualizar una transacción en la base de datos.

Campos serializados

Campo

id

id_proveedor

amount

currency

payment_stat
e

Validaciones implementadas

Validación del importe

Se comprueba que:

- exista un importe válido,
- y que el valor sea mayor que cero.

Esto evita registrar pagos inválidos o negativos.

Validación de la moneda

El sistema verifica que:

- la moneda no contenga números,
- y no supere el tamaño máximo permitido.

Validación del estado del pago

Los únicos estados permitidos son:

- pending

- completed
- failed
- refunded

Cualquier otro valor es rechazado automáticamente.

Validación general

También se comprueba la existencia del proveedor asociado a la transacción antes de procesar la operación.

IncidenceSerializer

El serializer IncidenceSerializer se utiliza para gestionar las incidencias relacionadas con las transacciones.

Gracias a este serializer es posible registrar errores, devoluciones o problemas técnicos asociados a los pagos realizados.

Campos serializados

Campo

id

id_transaction

description

type

Validaciones implementadas

Validación de la descripción

Se controla que la descripción:

- no supere el límite máximo de caracteres establecido.

Validación del tipo de incidencia

Los tipos de incidencias permitidos son:

- nonpayment
- connection_error
- devolution
- timeout
- authentication_error

Esto permite mantener una clasificación homogénea de los errores registrados en el sistema.

Validación general

Antes de registrar la incidencia, el sistema verifica que la transacción asociada exista correctamente en la base de datos.

Views

Las vistas del proyecto han sido desarrolladas utilizando Django REST Framework con el objetivo de gestionar la lógica principal de la API y controlar las operaciones relacionadas con proveedores, transacciones e incidencias.

Para la implementación se han utilizado:

- ModelViewSet para las operaciones CRUD automáticas
- Vistas personalizadas mediante `@api_view`
- Integración directa con la API de Stripe
- Respuestas HTTP estructuradas utilizando Response y códigos de estado

La lógica del sistema se divide en tres grandes bloques:

- Gestión de proveedores
- Gestión de pagos
- Gestión de devoluciones e incidencias

Gestión de proveedores

La gestión de proveedores se realiza mediante la clase `ProviderModelViewSet`.

Esta vista permite realizar automáticamente operaciones CRUD sobre los proveedores registrados en el sistema:

- Crear proveedores
- Consultar proveedores
- Actualizar información
- Eliminar registros

Funcionalidades implementadas

Método HTTP	Función
GET	Obtener proveedores

POST	Crear proveedor
PUT/PATCH	Actualizar proveedor
DELETE	Eliminar proveedor

El sistema utiliza JsonSerializer para validar y serializar la información antes de interactuar con la base de datos.

Gestión de transacciones

La gestión de transacciones se implementa mediante TransactionModelViewSet y varias vistas personalizadas encargadas del procesamiento de pagos con Stripe.

TransactionModelViewSet

Esta vista proporciona las operaciones CRUD básicas para las transacciones registradas en la base de datos.

Creación de pagos con Stripe

El endpoint create_payment permite iniciar una sesión de pago utilizando Stripe Checkout.

Cuando el cliente realiza una petición:

1. Se obtiene el proveedor asociado,
2. Se valida la información recibida,
3. Se crea una sesión de pago en Stripe,
4. Se registra una transacción inicial con estado pending.

Flujo de creación del pago

Cliente/Postman



create_payment



Stripe Checkout Session



Registro en SQLite

Datos almacenados

Durante la creación de la transacción se registran:

- Importe
- Moneda
- Proveedor asociado

- Estado del pago
- Identificador de sesión de Stripe

Manejo de errores

El sistema contempla diferentes excepciones:

- Proveedor inexistente
- Errores de Stripe
- Datos inválidos
- Excepciones generales

En caso de error, la API devuelve respuestas HTTP adecuadas junto con mensajes descriptivos.

Actualización automática mediante Webhooks

Para actualizar automáticamente el estado de las transacciones se ha implementado un webhook de Stripe mediante la vista `stripe_webhook`.

Los webhooks permiten que Stripe envíe eventos automáticamente al backend cuando ocurre una acción relacionada con un pago.

Funcionamiento del webhook

Cuando Stripe confirma que un pago ha sido completado:

1. Stripe envía un evento al endpoint del webhook.
2. El backend valida la firma del evento utilizando `STRIPE_WEBHOOK_SECRET`.
3. Se localiza la transacción correspondiente.
4. El estado pasa de pending a completed.
5. Se almacena el `payment_intent` devuelto por Stripe.

Flujo del webhook

Stripe



`stripe_webhook`



Validación del evento



Actualización de transacción

La implementación del webhook permite automatizar completamente la sincronización entre Stripe y la base de datos del proyecto.

Gestión de devoluciones

El sistema incorpora una funcionalidad para realizar devoluciones de pagos mediante la vista `refund_payment`.

Esta operación únicamente puede ejecutarse sobre transacciones cuyo estado sea completed.

Proceso de devolución

1. Se busca la transacción en la base de datos.
2. Se comprueba que el pago haya sido completado.
3. Se verifica la existencia del payment_intent.
4. Se solicita la devolución a Stripe.
5. El estado de la transacción cambia a refunded.
6. Se crea automáticamente una incidencia asociada.

Registro de incidencias

Cuando se realiza una devolución, el sistema genera automáticamente un registro en el modelo Incidence para mantener trazabilidad sobre la operación realizada.

Gestión de incidencias

La gestión de incidencias se realiza mediante IncidenceModelViewSet.

Esta vista permite:

- Consultar incidencias,
- Registrar errores manualmente
- Actualizar incidencias
- Eliminar registros

Las incidencias facilitan el seguimiento de problemas relacionados con:

- Errores de autenticación
- Fallos de conexión
- Devoluciones
- Impagos
- Tiempos de espera excedidos

Integración con Stripe

La integración con Stripe se realiza utilizando la librería oficial de Python.

La clave privada de Stripe se carga de forma segura desde variables de entorno almacenadas en un archivo .env e importadas posteriormente desde settings.py.

Gracias a esta integración, el sistema puede:

- Crear sesiones de pago

- Validar eventos webhook
- Gestionar devoluciones
- Sincronizar automáticamente los estados de las transacciones

Rutas y endpoints

Método	Endpoint	Descripción
GET	/api/incidence/	Listar incidencias
POST	/api/incidence/	Crear incidencias
GET	/api/incidence/{id}/	Listar incidencia por id
PUT	/api/incidence/{id}/	Actualizar incidencia
PATCH	/api/incidence/{id}/	Actualización parcial de la incidencia
DELETE	/api/incidence/{id}/	Eliminar incidencia
GET	/api/provider/	Listar proveedores
POST	/api/provider/	Crear proveedor
GET	/api/provider/{id}/	Listar proveedor por id
PUT	/api/provider/{id}/	Actualizar proveedor
PATCH	/api/provider/{id}/	Actualización parcial del proveedor
DELETE	/api/provider/{id}/	Eliminar proveedor
GET	/api/transaction/	Listar transacciones
POST	/api/transaction/	Crear transacción
GET	/api/transaction/{id}/	Listar transacción por id
PUT	/api/transaction/{id}/	Actualizar transacción
PATCH	/api/transaction/{id}/	Actualización parcial de la transacción
DELETE	/api/transaction/{id}/	Eliminar transacción
POST	/create-payment/	Crea una transacción
POST	/refund-payment/	Realiza devolución de una

		transacción completada
POST	/webhook/	Actualización parcial de la transacción al completar el pago con stripe

8 Pruebas realizadas

Durante el desarrollo del proyecto se realizaron diferentes pruebas funcionales con el objetivo de verificar el correcto funcionamiento de la API REST y garantizar la estabilidad del sistema.

Las pruebas permitieron validar:

- Funcionamiento de los endpoints
- Integración con Stripe
- Validaciones implementadas
- Manejo de errores
- Correcto almacenamiento de información en la base de datos

Para realizar las pruebas se utilizó principalmente Postman, enviando peticiones HTTP a los distintos endpoints desarrollados en la aplicación

Pruebas con Postman

Postman se utilizó como herramienta principal para comprobar el comportamiento de la API durante todas las fases del desarrollo.

Mediante esta herramienta fue posible:

- Enviar peticiones HTTP
- Comprobar respuestas JSON
- Verificar códigos de estado
- Simular diferentes escenarios de uso

Las pruebas realizadas incluyeron operaciones CRUD completas sobre:

- Proveedores
- Transacciones
- Incidencias

Además, Postman permitió probar el flujo completo de integración con Stripe.

Validación de errores

El sistema incorpora manejo de excepciones y validaciones personalizadas para controlar errores durante el procesamiento de datos.

Las validaciones se implementaron tanto en:

- Modelos (clean())
- Serializers de Django REST Framework.

Errores controlados

Entre los errores validados destacan:

- Proveedores inexistentes
- Transacciones inexistentes
- Datos vacíos
- Importes negativos
- Estados inválidos
- Errores de Stripe
- Devoluciones no permitidas

Cuando ocurre un error, la API devuelve:

- Un mensaje descriptivo
- Código HTTP correspondiente.

9 Problemas encontrados y soluciones

Durante el desarrollo del proyecto surgieron diferentes problemas técnicos relacionados con la integración de Stripe, la validación de datos y la sincronización de transacciones.

La resolución de estos problemas permitió mejorar la estabilidad, seguridad y funcionamiento general de la API REST.

Problemas relacionados con Stripe

Uno de los primeros problemas encontrados fue la configuración de la conexión con Stripe y la comprensión del funcionamiento de su API.

La integración requería:

- Claves privadas
- Configuración del entorno sandbox
- Creación de sesiones Checkout
- Validación de webhooks

Solución aplicada

Para solucionar este problema se consultó la documentación oficial de Stripe y se implementó:

- Uso de la librería oficial de Python
- Almacenamiento seguro de claves mediante .env
- Configuración centralizada desde settings.py

Esto permitió establecer una conexión segura entre el backend y Stripe

Problemas con los webhooks

Inicialmente las transacciones permanecían en estado pending incluso después de completar el pago en Stripe.

El problema estaba relacionado con la recepción y validación de eventos webhook.

Solución aplicada

Se implementó un endpoint específico para recibir eventos enviados automáticamente por Stripe.

Además:

- Se validó la firma del webhook mediante STRIPE_WEBHOOK_SECRET
- Se identificó correctamente la sesión Stripe
- Se actualizó automáticamente el estado de la transacción a completed.

Gracias a ello se consiguió sincronizar correctamente Stripe con la base de datos local.

Problemas con validaciones de datos

Durante las pruebas se detectó que algunos campos podían recibir:

- Valores vacíos
- Datos manipulados
- Estados inválidos
- Importes incorrectos

Esto podía provocar inconsistencias en la base de datos.

Solución aplicada

Se implementaron validaciones personalizadas tanto en:

- Modelos (clean())
- Serializers

Las validaciones permitieron:

- Restringir opciones válidas
- Controlar tamaños máximos
- Verificar relaciones existentes
- Devolver mensajes de error descriptivos

Problemas con las devoluciones

En las primeras versiones del sistema era posible intentar realizar devoluciones sobre transacciones que todavía no habían sido completadas correctamente.

Solución aplicada

Se añadió una validación previa antes de ejecutar la devolución:

- Comprobando que el estado fuese completed
- Verificando la existencia del payment_intent

Esto permitió evitar operaciones inválidas y mejorar la lógica del sistema.

Aprendizajes obtenidos

La resolución de estos problemas permitió adquirir conocimientos relacionados con:

- Integración de APIs externas
- Gestión de pagos online
- Arquitectura REST
- Validaciones backend
- Manejo de excepciones
- Organización profesional de proyectos Django

También se reforzaron habilidades relacionadas con:

- Lectura de documentación técnica
- Depuración de errores
- Pruebas de APIs
- Control de versiones mediante GitHub

10 Conclusiones

El desarrollo de este proyecto ha permitido implementar una API REST funcional para la gestión de pagos electrónicos utilizando Django REST Framework y Stripe como pasarela de pago principal.

A lo largo del proyecto se han aplicado conocimientos relacionados con:

- Desarrollo backend en Python
- Arquitectura REST
- Bases de datos relacionales
- Validaciones de datos
- Integración de APIs externas
- Control de versiones mediante Git y GitHub

El sistema desarrollado permite:

- Gestionar proveedores de pago
- Registrar transacciones
- Procesar pagos mediante Stripe
- Gestionar devoluciones
- Registrar incidencias relacionadas con las operaciones realizadas

Además, se ha conseguido implementar una arquitectura modular y organizada, facilitando el mantenimiento y la escalabilidad del proyecto.

Objetivos alcanzados

Los objetivos principales definidos al inicio del proyecto se han cumplido satisfactoriamente.

Entre los resultados obtenidos destacan:

- Creación de una API REST funcional
- Integración correcta con Stripe Checkout
- Actualización automática de pagos mediante webhooks
- Implementación de validaciones personalizadas
- Gestión de incidencias
- Almacenamiento persistente de datos utilizando SQLite

También se consiguió aplicar buenas prácticas de desarrollo como:

- Uso de variables de entorno
- Separación modular de componentes
- Organización del código
- Documentación técnica del proyecto

Aprendizajes obtenidos

El proyecto ha servido para reforzar conocimientos técnicos relacionados con el desarrollo backend profesional.

Entre los principales aprendizajes destacan:

- Uso avanzado de Django y Django REST Framework
- Diseño de modelos relacionales
- Implementación de APIs RESTful

- Integración con servicios externos
- Manejo de peticiones HTTP
- Validación de datos
- Gestión de errores

Además, se adquirieron conocimientos prácticos sobre:

- Funcionamiento de Stripe
- Procesamiento de pagos online
- Uso de webhooks
- Sincronización de eventos externos con la base de datos

El desarrollo del proyecto también permitió mejorar habilidades relacionadas con:

- Planificación de tareas
- Organización mediante Trello
- Pruebas con Postman
- Control de versiones con GitHub

Posibles mejoras futuras

Aunque el sistema desarrollado cumple correctamente los objetivos planteados, existen diversas mejoras que podrían implementarse en futuras versiones del proyecto.

Entre las posibles ampliaciones destacan:

- Soporte para múltiples proveedores de pago además de Stripe
- Sistema de logs y monitorización
- Despliegue en un entorno de producción.

11 Bibliografía

- [Documentación de Python](#)
- [Documentación de Django](#)
- [Documentación de Django REST Framework](#)
- [Documentación de Stripe](#)
- [Documentación de Postman](#)
- [Documentación de Python SDK](#)
- [Documentación de SQLite](#)